

Cache-Aside 缓存策略数据一致性解题过程

题目原文

试题信息

阅读下列关于数据库设计的说明，回答问题1至问题3。

说明原文

为了提升某购物网站的用户使用体验，采用数据库缓存技术将经常访问的商品信息存储在一个临时高速数据存储层中，这个临时存储层使得未来对这些数据的请求响应比通过访问主数据库更快。为了确保快速、及时获取最新数据，项目组决定采用缓存数据技术开展项目设计，经讨论，决定采用 Cache-Aside 策略作为缓存手段。

问题原文

问题1 (10分)

使用 Cache-Aside 缓存策略，当用户查询商品信息时，此请求会从应用程序服务器发送请求到数据库服务器，通过缓存系统处理，然后返回所请求的信息，请补充完善图1中 (1) ~ (5) 处的内容，协助设计师完成缓存系统中的数据读取模块的设计方案。

问题2 (6分)

使用 Cache-Aside 缓存策略，当用户更新商品信息时，此请求会从应用服务器发至数据库服务器，并进行缓存处理，请补充完善图2中 (1) 和 (2) 处的内容，协助工程师完成缓存系统中的数据更新模块的设计方案。

问题3 (9分)

李工设计了多线程并发数据访问方案，采用线程1执行数据更新，线程2执行数据读取。王工认为这样的设计方案会存在数据不一致现象，请说明李工所设计的方案为什么会出现数据库与缓存中的数据不一致的情况，并给出3种或以上的解决方案。

题图说明

图1描述 Cache-Aside

策略下的数据读取流程，重点是“先查缓存，缓存未命中再查数据库，并将数据库结果写回缓存”。

图2描述 Cache-Aside 策略下的数据更新流程，重点是“先更新数据库，再删除缓存”，而不是直接更新缓存。

题图

图1 数据读取模块

Cache-Aside 数据读取模块

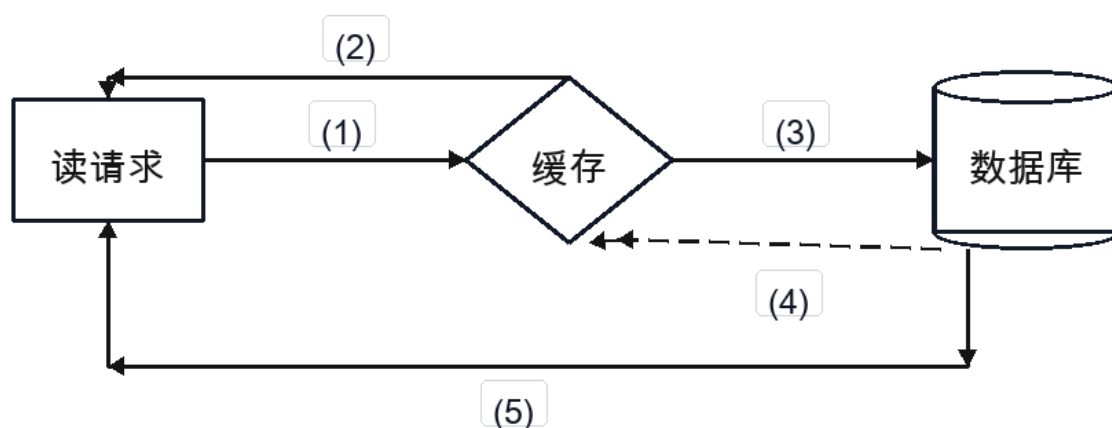


图1 数据库缓存系统中的数据读取模块的设计方案

图2 数据更新模块

Cache-Aside 数据更新模块

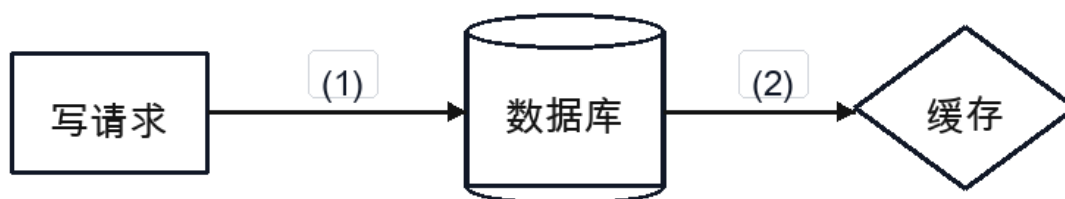


图2 数据库缓存系统中的数据更新模块的设计方案

题目结论

问题1

图1中 (1) ~ (5) 应填写 :

- (1) : 查询缓存 / 读取缓存
- (2) : 缓存命中, 返回数据
- (3) : 缓存未命中, 查询数据库
- (4) : 将数据库查询结果写入缓存
- (5) : 返回数据库查询结果

问题2

图2中 (1) 和 (2) 应填写 :

- (1) : 更新数据库
- (2) : 删除缓存 / 使缓存失效

问题3

出现不一致的典型原因：

线程2读取时缓存未命中，先从数据库读到旧值；此时线程1更新数据库并删除缓存；随后线程2又把旧值写回缓存，导致数据库中是新值。

可选解决方案：

- 延迟双删
- 缓存删除失败重试
- 消息队列或 binlog 订阅异步删除缓存
- 对同一商品加互斥锁或分布式锁
- 设置缓存过期时间
- 使用版本号或时间戳避免旧值回填
- 强一致场景改用同步更新或不使用缓存

解题思路

问题1：Cache-Aside 读取流程

Cache-Aside 又称旁路缓存模式。

读取数据时，应用程序负责控制缓存和数据库访问。

标准流程是：

1. 应用先查询缓存。
2. 如果缓存命中，直接返回缓存数据。
3. 如果缓存未命中，再查询数据库。
4. 从数据库读到数据后，写入缓存。
5. 将查询结果返回给用户。

对应图1：

- (1)：读请求访问缓存，即查询缓存。
- (2)：缓存命中后，缓存直接返回数据。
- (3)：缓存未命中后，继续查询数据库。
- (4)：数据库查询结果回填到缓存。
- (5)：数据库查询结果返回给读请求。

标准答法：

- (1)：查询缓存
- (2)：缓存命中，返回数据
- (3)：缓存未命中，查询数据库
- (4)：将数据库查询结果写入缓存
- (5)：返回数据库查询结果

问题2：Cache-Aside 更新流程

Cache-Aside 更新数据时，通常不直接更新缓存，而是：

先更新数据库，再删除缓存。

这样做的原因是：

- 数据库是权威数据源；
- 删除缓存后，下一次读取会缓存未命中，再从数据库读取最新值并回填缓存。

对应图2：

- (1)：写请求更新数据库。
- (2)：数据库更新成功后，删除缓存或使缓存失效。

标准答法：

- (1)：更新数据库
- (2)：删除缓存 / 使缓存失效

问题3：为什么会出现数据不一致

多线程并发时，读线程和写线程的执行顺序可能交错。

典型不一致时序如下：

1. 线程2读取商品信息，发现缓存未命中。
2. 线程2查询数据库，读到旧值 A。
3. 线程1更新数据库，将商品信息改为新值 B。
4. 线程1删除缓存。
5. 线程2把之前读到的旧值 A 写入缓存。

最终结果：

数据库 = 新值 B
缓存 = 旧值 A

这样就出现了数据库与缓存数据不一致。

除此之外，下面情况也可能导致不一致：

- 数据库更新成功，但删除缓存失败。
- 主从数据库复制延迟，缓存回填了从库旧数据。
- 多个线程同时缓存未命中，旧请求后写入覆盖新请求结果。
- 缓存过期时间过长，脏数据长期存在。

解决方案

方案1：延迟双删

流程：

先删除缓存
再更新数据库
等待一小段时间
再次删除缓存

作用：

第二次删除可以清理并发读请求回填的旧缓存。

方案2：更新数据库后删除缓存，并保证删除成功

Cache-Aside 常用写法：

更新数据库
删除缓存

如果删除缓存失败，应通过重试机制保证最终删除。

可使用：

本地重试
任务表
消息队列
失败补偿任务

方案3：使用消息队列或 binlog 订阅删除缓存

数据库更新后，发送缓存失效消息。

也可以订阅数据库 binlog，监听商品表变化，然后异步删除对应缓存。

优点：

解耦业务更新和缓存删除；
可重试；
适合分布式系统。

方案4：对同一商品加互斥锁或分布式锁

对同一个商品 ID 的更新和缓存回填加锁。

目的：

避免多个线程同时读写同一商品缓存，防止旧值覆盖新值。

适合热点商品、库存、价格等一致性要求较高的数据。

方案5：设置合理缓存过期时间

即使出现短时间不一致，缓存也会在 TTL 到期后失效。

这种方案不能彻底避免不一致，但可以限制不一致持续时间。

方案6：使用版本号或时间戳

缓存数据中携带版本号或更新时间。

回填缓存前检查：

如果待写入数据版本较旧，则拒绝写入缓存。

这样可以避免旧查询结果覆盖新数据。

方案7：强一致场景减少或绕过缓存

对于价格、库存、支付状态等强一致数据，可以：

- 不缓存。
- 缩短缓存时间。

- 使用同步更新策略。
- 使用数据库事务和行锁保证一致性。

标准答题模板

问题1答题模板

- (1) 查询缓存；
- (2) 缓存命中，返回数据；
- (3) 缓存未命中，查询数据库；
- (4) 将数据库查询结果写入缓存；
- (5) 返回数据库查询结果。

问题2答题模板

- (1) 更新数据库；
- (2) 删除缓存或使缓存失效。

问题3答题模板

并发读写时可能出现时序交错。线程2缓存未命中后从数据库读到旧值，线程1随后更新数据库并删除缓存，线程2再将旧值写回缓存，导致数据不一致。

解决方案包括：采用延迟双删；数据库更新后删除缓存并进行失败重试；通过消息队列或 binlog 订阅异步删除缓存；对同一商品加互斥锁。

关键词速记

Cache-Aside 读：先缓存，未命中查库，查库后回填缓存。

Cache-Aside 写：先更新数据库，再删除缓存。

不一致原因：并发读旧值，写线程删缓存，读线程旧值回填。

解决方案：延迟双删、删除重试、MQ/binlog、互斥锁、TTL、版本号。

易错点

1. Cache-Aside 更新时通常是“删除缓存”，不是“更新缓存”。
2. 先更新数据库再删除缓存，是为了让下一次读取从数据库加载最新值。
3. 删除缓存失败也会导致缓存旧值长期存在。
4. 延迟双删的重点是处理并发读导致的旧值回填。
5. TTL 只能缩短不一致时间，不能彻底保证强一致。